

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1
_____o0o_____



BÀI TẬP LỚN CƠ SỞ DỮ LIỆU PHÂN TÁN

Tên đề tài: Hệ thống đăng ký học phần nhiều cơ sở

NHÓM LỚP : 10

Trần Nguyễn Nhật Quang	MSSV: B23DCCN694
Phạm Khánh Toàn	MSSV: B23DCCN834
Nguyễn Quang Huy	MSSV: B23DCCN400

Giảng viên hướng dẫn: TS. Kim Ngọc Bách

HÀ NỘI, 06/2026

Mục lục

1	TỔNG QUAN VÀ MÔ TẢ BÀI TOÁN	4
1.1	Bối cảnh hệ thống	4
1.2	Mục tiêu đề án	4
1.3	Yêu cầu chức năng của hệ thống	4
2	THIẾT KẾ LƯỢC ĐỒ TOÀN CỤC	6
2.1	Phân tích các thực thể và mối quan hệ	6
2.2	Lược đồ cơ sở dữ liệu toàn cục	6
2.3	Lược đồ mô hình quan hệ	10
3	PHÂN MẢNH VÀ CẤP PHÁT DỮ LIỆU PHÂN TÁN	11
3.1	Đề xuất chiến lược phân mảnh dữ liệu	11
3.2	Thiết kế mảnh cục bộ	11
3.3	Thiết kế mảnh dùng chung / Dữ liệu sao chép	12
3.4	Mô hình cấp phát dữ liệu	13
4	XỬ LÝ GIAO DỊCH VÀ KIỂM SOÁT ĐỒNG THỜI	15
4.1	Bài toán tranh chấp dữ liệu (Lỗi Overbooking)	15
4.2	Cơ chế xử lý đăng ký đồng thời	15
4.2.1	Sử dụng Transaction và khóa dòng (FOR UPDATE)	15
4.2.2	Cơ chế phòng chống Deadlock	16
4.3	Minh họa kịch bản và kết quả mô phỏng	16
5	TRUY VẤN PHÂN TÁN	18
5.1	Cơ chế thực thi truy vấn phân tán của hệ thống	18
5.2	Các truy vấn phân tán	18
5.2.1	Thống kê số lượng sinh viên đăng ký học phần theo từng cơ sở	18
5.2.2	Tìm kiếm học phần có nhiều sinh viên đăng ký nhất toàn trường	19
5.2.3	Trích xuất danh sách sinh viên đăng ký chéo cơ sở	19
5.2.4	Tính toán tỷ lệ lấp đầy của các lớp học phần toàn hệ thống	19
5.2.5	Thống kê quy mô lớp học và số lượng sinh viên theo cơ sở	20
6	YÊU CẦU NÂNG CAO	21
6.1	Mô phỏng xung đột cập nhật và phòng tránh deadlock	21
6.1.1	Bối cảnh mô phỏng	21
6.1.2	Nguy cơ phát sinh lỗi dữ liệu	21
6.1.3	Cơ chế kiểm soát đồng thời	22
6.1.4	Vai trò của trigger trong kiểm soát số	22
6.1.5	Kết quả và nhận xét	23

6.2	Nhân bản dữ liệu dùng chung	23
6.2.1	Lý do cần nhân bản	23
6.2.2	Các bảng được nhân bản	24
6.2.3	Lợi ích của replication	24
6.2.4	Hạn chế và cách lựa chọn dữ liệu nhân bản	24
6.3	Cơ chế xử lý khi một site tạm thời mất kết nối	25
6.3.1	Bối cảnh lỗi site	25
6.3.2	Bảng OfflineOperationLog	25
6.3.3	Luồng xử lý khi site mất kết nối	26
6.3.4	Đánh giá cơ chế xử lý offline	26
6.4	Kết luận	27
7	TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG	28
7.1	Kiến trúc hệ thống và công nghệ sử dụng	28
7.2	Đánh giá tính đúng đắn và hiệu năng của hệ thống	30
7.3	Đánh giá xử lý lỗi và kịch bản kết nối	31

LỜI NÓI ĐẦU

Trong bối cảnh chuyển đổi số đang diễn ra mạnh mẽ trong lĩnh vực giáo dục, quy mô của các trường Đại học không ngừng được mở rộng với sự hình thành của nhiều cơ sở đào tạo trải dài trên các khu vực địa lý khác nhau. Sự phát triển này mang lại cơ hội tiếp cận giáo dục linh hoạt cho sinh viên, nhưng đồng thời cũng đặt ra một bài toán đầy thách thức cho công tác quản trị hệ thống thông tin.

Thực tế cho thấy, mô hình quản lý dữ liệu đào tạo tập trung tại một máy chủ (Centralized Database) truyền thống đang bộc lộ nhiều hạn chế nghiêm trọng. Khi lượng sinh viên tăng vọt, đặc biệt là vào các đợt cao điểm đăng ký học phần, hệ thống máy chủ đơn lẻ thường xuyên phải đối mặt với tình trạng quá tải về hiệu năng và rủi ro sập mạng. Những sự cố này không chỉ gây gián đoạn quy trình đào tạo mà còn ảnh hưởng trực tiếp đến trải nghiệm của sinh viên và công tác quản lý của nhà trường.

Đứng trước bài toán thực tiễn đó, việc chuyển đổi sang kiến trúc Hệ Cơ sở dữ liệu phân tán (Distributed Database System - DDBMS) là một xu thế tất yếu. Bằng cách phân tán dữ liệu đến các trạm (site) cục bộ nơi nó được sử dụng thường xuyên nhất, hệ thống có thể tối ưu hóa không gian lưu trữ, tăng cường tính sẵn sàng (High Availability), tính chịu lỗi và đẩy nhanh tốc độ truy xuất dữ liệu một cách vượt trội.

1 TỔNG QUAN VÀ MÔ TẢ BÀI TOÁN

1.1 Bối cảnh hệ thống

Hệ thống được đặt trong bối cảnh giả định về một trường đại học có quy mô lớn, sở hữu mạng lưới gồm năm cơ sở đào tạo phân bố rải rác tại các vị trí khác nhau bao gồm cơ sở Hòa Lạc, Ngọc Trục, Hà Đông, Cầu Giấy và Thành phố Hồ Chí Minh. Để đảm bảo hiệu quả vận hành thực tế, mỗi cơ sở này được thiết kế để hoạt động với mức độ tự trị rất cao, cho phép ban quản lý tại từng địa phương kiểm soát độc lập các luồng thông tin liên quan đến sinh viên, đội ngũ giảng viên, cơ sở vật chất phòng học và các lớp học phần được mở cục bộ. Mặc dù có sự phân cấp mạnh mẽ về mặt quản trị tại từng trạm, toàn bộ năm cơ sở đào tạo này vẫn được liên kết chặt chẽ và nhất quán thông qua việc sử dụng chung một khung chương trình đào tạo chung của nhà trường. Dựa trên cấu trúc mạng lưới này, trong mỗi kỳ đăng ký tín chỉ, hệ thống mang lại sự linh hoạt tối đa cho người học khi cho phép sinh viên trực tiếp đăng ký các môn học được tổ chức tại cơ sở quản lý chính của mình, hoặc thực hiện các nghiệp vụ đăng ký chéo sang những cơ sở đào tạo khác nhằm đáp ứng kịp thời nhu cầu học tập cá nhân.

1.2 Mục tiêu đề án

Hướng tới việc giải quyết các thách thức kỹ thuật đặt ra trong bối cảnh hệ thống quy mô lớn, đề án xác định rõ các mục tiêu cốt lõi bao trùm khía cạnh kiến trúc hạ tầng, xử lý giao dịch và trích xuất dữ liệu. Về mặt kiến trúc tổng thể, đề án tập trung xây dựng một hệ thống cơ sở dữ liệu phân tán hoàn chỉnh, được triển khai thực tế trên năm máy chủ độc lập tương ứng với năm cơ sở đào tạo nhằm đảm bảo tính phân tán vật lý của dữ liệu. Về khía cạnh kiểm soát giao dịch, mục tiêu quan trọng và mang tính kỹ thuật nhất của hệ thống là phải giải quyết triệt để tình trạng lỗi mất cập nhật cũng như các sự cố tranh chấp dữ liệu; đặc biệt là trong các tình huống thực tế khi có nhiều người dùng cùng lúc truy cập và thực hiện thao tác đăng ký vào một lớp học phần đang trong tình trạng sắp giới hạn chỗ ngồi. Cuối cùng, ở cấp độ tổng hợp thông tin quản lý, hệ thống phải đảm bảo khả năng cung cấp các báo cáo toàn trường thông qua việc áp dụng kỹ thuật truy vấn phân tán đa luồng, giúp ban quản trị có được cái nhìn tổng quan và chính xác từ toàn bộ các trạm cơ sở.

1.3 Yêu cầu chức năng của hệ thống

Để đáp ứng trọn vẹn các mục tiêu quản trị trong một kiến trúc phân tán, hệ thống được thiết kế để cung cấp một tổ hợp các nhóm chức năng, từ khâu lưu trữ cơ sở dữ liệu nền tảng đến việc xử lý các yêu cầu phức tạp. Nhóm chức năng đầu tiên tập trung vào công tác quản lý danh mục cốt lõi, bao gồm việc quản trị thông tin về mạng lưới các cơ sở đào tạo, hệ thống các khoa chuyên môn và một danh mục học phần chuẩn hóa được

dùng chung cho toàn trường. Song song với đó, hệ thống trang bị bộ công cụ quản lý tài nguyên cục bộ tại từng node mạng, cho phép mỗi cơ sở điều hành độc lập và chặt chẽ hồ sơ sinh viên, đội ngũ giảng viên, cũng như quản lý cơ sở vật chất phòng học và sắp xếp lịch học thực tế. Sự kết hợp này đảm bảo hệ thống vừa có tính thống nhất trên toàn trường, vừa duy trì được sự tự trị linh hoạt tại từng địa phương. Đối với nhóm nghiệp vụ cốt lõi, hệ thống tập trung giải quyết bài toán vận hành đào tạo thông qua chức năng mở các lớp học phần, đi kèm với những thiết lập khắt khe về giới hạn sĩ số sinh viên nhằm phản ánh đúng sức chứa của phòng học. Dựa trên danh sách các lớp học phần đang mở, sinh viên được cấp quyền chủ động tham gia vào các giao dịch đăng ký môn học hoặc tiến hành hủy đăng ký nếu có sự điều chỉnh trong kế hoạch học tập cá nhân. Ngay sau khi hoàn tất các giao dịch tương tác này, hệ thống lập tức hỗ trợ sinh viên tra cứu kết quả ghi nhận và xem chi tiết thời khóa biểu cá nhân một cách trực quan. Cuối cùng, nhằm phục vụ công tác giám sát và ra quyết định chiến lược, hệ thống cung cấp một phân hệ báo cáo thống kê cấp cao, hiển thị dưới dạng bảng điều khiển (dashboard) dành riêng cho Ban giám hiệu. Phân hệ này có khả năng tổng hợp dữ liệu từ toàn bộ năm cơ sở, cho phép theo dõi và xuất các báo cáo thống kê chi tiết theo từng cơ sở đơn lẻ cũng như bức tranh tổng thể trên quy mô toàn hệ thống đại học.

2 THIẾT KẾ LƯỢC ĐỒ TOÀN CỤC

2.1 Phân tích các thực thể và mối quan hệ

Dựa trên phân tích yêu cầu, hệ thống bao gồm 11 thực thể (Entity) có mối quan hệ chặt chẽ với nhau, phục vụ cho quy trình quản lý tài nguyên và nghiệp vụ đăng ký học phần:

1. **Cơ sở (CoSo):** Quản lý thông tin các cơ sở đào tạo phân tán của trường đại học.
2. **Khoa (Khoa):** Quản lý thông tin các khoa chuyên môn chung của toàn trường.
3. **Học phần (HocPhan):** Lưu trữ thông tin danh mục các môn học được mở, chịu sự quản lý của một Khoa cụ thể.
4. **Chương trình đào tạo (ChuongTrinhDaoTao):** Quy định khung chương trình cho từng khoa, ánh xạ các học phần vào học kỳ dự kiến và bao gồm các môn học.
5. **Đợt đăng ký (DotDangKy):** Cấu hình khoảng thời gian cho phép đăng ký tín chỉ đối với từng nhóm đối tượng (Khoa, Khóa nhập học).
6. **Phòng học (PhongHoc):** Quản lý không gian học tập (tên phòng, sức chứa) trực thuộc tại một Cơ sở.
7. **Giảng viên (GiangVien):** Quản lý hồ sơ nhân sự của giảng viên, thuộc một Khoa và được phân bổ công tác tại một Cơ sở.
8. **Sinh viên (SinhVien):** Quản lý thông tin cá nhân của người học, trực thuộc một Khoa và chịu sự quản lý của một Cơ sở cố định.
9. **Lớp học phần (LopHocPhan):** Thực thể cốt lõi để sinh viên thực hiện đăng ký. Lớp học phần được mở cho một Học phần, do một Giảng viên phụ trách, tổ chức tại một Cơ sở cụ thể và có giới hạn khắt khe về sức chứa.
10. **Lịch học (LichHoc):** Xác định lịch trình chi tiết (không gian và thời gian: thứ, tiết bắt đầu, tiết kết thúc, phòng học) cho từng Lớp học phần.
11. **Đăng ký (DangKy):** Thực thể lưu trữ tác vụ đăng ký tín chỉ. Thể hiện mối quan hệ nhiều - nhiều giữa Sinh viên và Lớp học phần nằm trong phạm vi của một Đợt đăng ký cụ thể.

2.2 Lược đồ cơ sở dữ liệu toàn cục

Lược đồ cơ sở dữ liệu được thiết kế chi tiết với 11 bảng toàn cục. Dưới đây là mô tả cấu trúc đầy đủ của từng bảng, bao gồm các khóa chính (PK), khóa ngoại (FK), các trường dữ liệu chi tiết cùng với các ràng buộc theo sát mã nguồn thực tế:

1. Bảng CoSo (Cơ sở)

- ID (PK): Mã cơ sở.
- name_headquarter: Tên cơ sở.
- address: Địa chỉ cơ sở.

2. Bảng Khoa (Khoa)

- ID (PK): Mã khoa.
- name_department: Tên khoa.

3. Bảng HocPhan (Học phần)

- ID (PK): Mã học phần.
- name_subject: Tên môn học.
- number_of_credit: Số tín chỉ.
- ID_department (FK): Mã khoa quản lý (tham chiếu Khoa.ID).

4. Bảng ChuongTrinhDaoTao (Chương trình đào tạo)

- ID_department (PK, FK): Mã khoa quản lý (tham chiếu Khoa.ID).
- ID_subject (PK, FK): Mã học phần (tham chiếu HocPhan.ID).
- suggested_semester: Học kỳ dự kiến/đề xuất (int).
- is_required: Môn học bắt buộc (boolean, mặc định true).

5. Bảng DotDangKy (Đợt đăng ký)

- ID (PK): Mã đợt đăng ký.
- semester: Học kỳ (int).
- school_year: Năm học (int).
- ID_department (FK): Mã khoa áp dụng (tham chiếu Khoa.ID).
- admission_year: Khóa/Năm nhập học áp dụng đợt đăng ký.
- start_time: Thời gian hệ thống bắt đầu mở cổng (timestamp).
- end_time: Thời gian đóng cổng (timestamp).
- is_open: Trạng thái mở/đóng thủ công (boolean, mặc định true).
- description: Mô tả thông tin đợt đăng ký.
- *Ràng buộc Check*: end_time > start_time.

6. Bảng PhongHoc (Phòng học)

- ID (PK): Mã phòng học.
- name_room: Tên phòng học.
- capacity: Sức chứa của phòng.
- ID_headquarter (FK): Mã cơ sở chứa phòng học (tham chiếu CoSo.ID).

7. Bảng GiảngVien (Giảng viên)

- ID (PK): Mã giảng viên.
- name_teacher: Họ và tên giảng viên.
- address_teacher: Địa chỉ giảng viên.
- degree: Học vị / Trình độ.
- phone_teacher: Số điện thoại.
- ID_department (FK): Mã khoa trực thuộc (tham chiếu Khoa.ID).
- ID_headquarter (FK): Mã cơ sở công tác (tham chiếu CoSo.ID).

8. Bảng SinhVien (Sinh viên)

- ID (PK): Mã sinh viên.
- name_student: Họ và tên sinh viên.
- date_of_birth: Ngày sinh.
- address_student: Địa chỉ của sinh viên.
- formal_class: Lớp hành chính.
- year_of_admission: Năm nhập học.
- phone_student: Số điện thoại.
- ID_department (FK): Mã khoa trực thuộc (tham chiếu Khoa.ID).
- ID_headquarter (FK): Mã cơ sở quản lý (tham chiếu CoSo.ID).

9. Bảng LopHocPhan (Lớp học phần)

- ID (PK): Mã lớp học phần.
- semester: Học kỳ.
- school_year: Năm học.

- number_of_student: Số lượng sinh viên đã đăng ký (mặc định là 0).
- max_student: Số lượng sinh viên tối đa của lớp học.
- ID_subject (FK): Mã học phần (tham chiếu HocPhan.ID).
- ID_teacher (FK): Mã giảng viên giảng dạy (tham chiếu GiangVien.ID).
- ID_headquarter (FK): Mã cơ sở mở lớp học phần (tham chiếu CoSo.ID).
- Ràng buộc Check: Sĩ số (number_of_student) không được bé hơn 0 và không được vượt quá sĩ số tối đa (max_student).

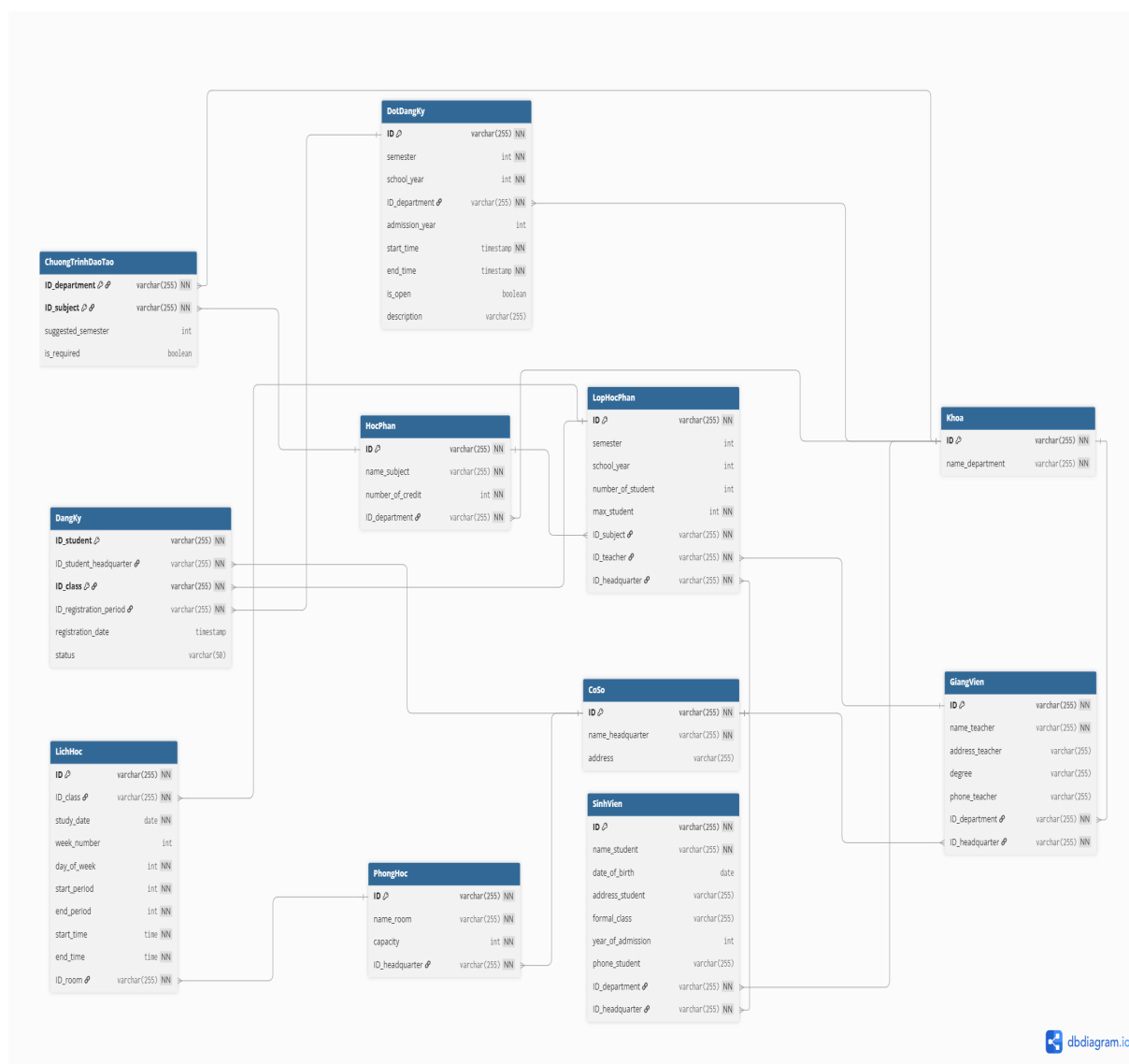
10. Bảng LichHoc (Lịch học)

- ID (PK): Mã lịch học.
- ID_class (FK): Mã lớp học phần (tham chiếu LopHocPhan.ID).
- day_of_week: Thứ trong tuần.
- study_date: Lịch học.
- week_number: Số tuần.
- start_time: Thời gian bắt đầu.
- end_time: Thời gian kết thúc.
- start_period: Tiết bắt đầu.
- end_period: Tiết kết thúc.
- ID_room (FK): Mã phòng học (tham chiếu PhongHoc.ID).
- Ràng buộc Check: day_of_week phải nằm từ thứ 2 đến thứ 8 (chủ nhật);
start_period > 0 và end_period >= start_period.

11. Bảng DangKy (Đăng ký học phần - Đã cập nhật trường dữ liệu)

- ID_student (PK): Mã sinh viên.
- ID_class (PK, FK): Mã lớp học phần (tham chiếu LopHocPhan.ID).
- ID_student_headquarter (FK): Mã cơ sở của sinh viên (tham chiếu CoSo.ID).
- ID_registration_period (FK): Mã đợt đăng ký (tham chiếu DotDangKy.ID).
- registration_date: Thời gian thực hiện đăng ký.
- status: Trạng thái đăng ký.

2.3 Lược đồ mô hình quan hệ



3 PHÂN MẢNH VÀ CẤP PHÁT DỮ LIỆU PHÂN TÁN

3.1 Đề xuất chiến lược phân mảnh dữ liệu

Hệ thống áp dụng chiến lược phân tán dữ liệu hỗn hợp, kết hợp giữa phương pháp phân mảnh ngang nguyên thủy cho các dữ liệu giao dịch cục bộ và kỹ thuật sao chép toàn phần đối với các dữ liệu danh mục dùng chung. Chiến lược thiết kế này được định hình dựa trên nguyên tắc cốt lõi là tối đa hóa tính cục bộ của tham chiếu (locality of reference), đảm bảo rằng thông tin luôn được lưu trữ tại vị trí vật lý gần nhất với nơi nó thường xuyên được truy xuất. Đối với những luồng dữ liệu gắn liền với hoạt động vận hành thường nhật của từng cơ sở, hệ thống sẽ chia nhỏ cơ sở dữ liệu toàn cục thành các phân đoạn và phân bổ về các trạm tương ứng. Ngược lại, đối với nhóm dữ liệu nền tảng mang tính chất tra cứu toàn trường, cơ chế nhân bản sẽ được kích hoạt để đảm bảo người dùng có thể đọc dữ liệu ngay lập tức mà không cần phụ thuộc vào đường truyền mạng. Lựa chọn kiến trúc này không chỉ triệt tiêu tối đa độ trễ trong các thao tác truy vấn mạng diện rộng, mà còn giúp từng cơ sở đào tạo duy trì khả năng hoạt động tự trị ngay cả khi kết nối đến máy chủ trung tâm hoặc các node khác gặp sự cố gián đoạn.

3.2 Thiết kế mảnh cục bộ

Để hiện thực hóa chiến lược lưu trữ cục bộ, hệ thống lựa chọn kỹ thuật phân mảnh ngang (Horizontal Fragmentation) làm phương pháp chủ đạo. Tiêu chí lõi để tiến hành phân tách dữ liệu được xác định dựa trên thuộc tính mã cơ sở, được định nghĩa là trường dữ liệu `ID_headquarter` trong lược đồ cơ sở dữ liệu. Theo quy tắc cấu hình này, bất kỳ bản ghi nào sở hữu giá trị khóa ngoại `ID_headquarter` tương ứng với một cơ sở nhất định sẽ được hệ thống bóc tách và lưu trữ độc quyền tại máy chủ vật lý của chính cơ sở đó. Dựa trên lược đồ toàn cục đã thiết kế, các bảng dữ liệu được áp dụng trực tiếp quy tắc phân mảnh ngang nguyên thủy bao gồm Sinh viên, Giảng viên, Phòng học và Lớp học phần. Đây là nhóm các thực thể tài nguyên chịu sự quản lý hành chính và vận hành trực tiếp bởi từng cơ sở đào tạo riêng biệt.

- **Các bảng áp dụng:** SinhVien, GiangVien, PhongHoc, LopHocPhan, LichHoc, DangKy.
- **Quy tắc phân mảnh nguyên thủy:** Dữ liệu chứa thuộc tính định vị `ID_headquarter` thuộc về cơ sở nào sẽ được phân mảnh và lưu trữ độc quyền tại Server Database của cơ sở đó (Ví dụ đối với sinh viên: `SELECT * FROM SinhVien WHERE ID_headquarter = 'HL'`). Các bảng tuân theo quy tắc này bao gồm SinhVien, GiangVien, PhongHoc, LopHocPhan.

- **Phân mảnh dẫn xuất (Derived Fragmentation):** Bảng DangKy và bảng LichHoc được phân mảnh dẫn xuất theo sự phân mảnh của bảng LopHocPhan (thông qua khóa ngoại ID_class). Việc thiết kế này đảm bảo toàn bộ giao dịch đăng ký tín chỉ của một lớp học phần và thông tin lịch trình của lớp đó luôn được xử lý đóng kín tại một Node duy nhất. Nhờ cách bố trí này, toàn bộ hồ sơ đăng ký và lịch trình giảng dạy của một lớp học phần bất kỳ sẽ luôn được đặt cùng một vị trí vật lý với chính lớp học phần đó. Cấu trúc đóng gói dữ liệu tại một node duy nhất giúp hệ thống loại bỏ triệt để các rủi ro về xung đột giao dịch phân tán, từ đó đảm bảo hiệu năng tối đa khi sinh viên thực hiện thao tác ghi nhận hoặc hủy đăng ký học phần. Đồng thời, việc bảng Đăng ký có chứa thêm trường ID_student_headquarter tham chiếu đến cơ sở gốc của sinh viên cũng tạo tiền đề quan trọng giúp tối ưu hóa hiệu suất cho các truy vấn kiểm tra tình trạng đăng ký chéo cơ sở.

3.3 Thiết kế mảnh dùng chung / Dữ liệu sao chép

Bên cạnh các mảnh dữ liệu cục bộ phục vụ công tác quản lý tài nguyên nội bộ, hệ thống áp dụng chiến lược nhân bản toàn phần (Full Replication) đối với nhóm dữ liệu danh mục mang tính chất dùng chung trên toàn trường. Cụ thể, năm bảng dữ liệu được hệ thống lựa chọn để tiến hành sao chép và phân bổ bản sao tới toàn bộ các trạm cơ sở bao gồm bảng CoSo, bảng Khoa, bảng ChuongTrinhDaoTao, bảng DotDangKy và bảng HocPhan. Đây là nhóm các thực thể đóng vai trò xây dựng cấu trúc nền tảng cho hệ thống, lưu trữ các thông tin tĩnh về mạng lưới cơ sở đào tạo, hệ thống khoa chuyên môn chung của toàn trường cũng như danh mục toàn diện của các môn học được phép tổ chức giảng dạy. Quyết định áp dụng cơ chế sao chép toàn phần cho ba thực thể này xuất phát từ việc phân tích sâu sắc đặc thù truy xuất và vòng đời của dữ liệu trong thực tế vận hành. Đặc điểm chung của các bảng danh mục này là chúng sở hữu tần suất thực thi các truy vấn đọc dữ liệu (Select) khổng lồ, đặc biệt tăng vọt trong những đợt cao điểm khi hàng ngàn sinh viên đồng loạt đăng nhập để tra cứu chương trình đào tạo và lên kế hoạch học tập. Ở chiều ngược lại, các thao tác làm thay đổi trạng thái dữ liệu như thêm mới, cập nhật hay xóa (Insert/Update/Delete) đối với thông tin cơ sở, khoa và học phần lại rất hiếm khi diễn ra. Bằng việc đặt một bản sao hoàn chỉnh của nhóm dữ liệu này tại tất cả các máy chủ vật lý, hệ thống giải quyết triệt để bài toán về độ trễ mạng. Khi sinh viên truy cập vào phân hệ đăng ký, ứng dụng có thể ngay lập tức truy xuất và hiển thị danh sách môn học từ cơ sở dữ liệu nội bộ thay vì phải gửi yêu cầu truy vấn băng qua mạng diện rộng tới một máy chủ trung tâm. Cách tiếp cận này không chỉ mang lại tốc độ phản hồi gần như tức thời, mà còn giúp giảm tải đáng kể lưu lượng truy cập trên đường truyền mạng lưới, qua đó bảo vệ hệ thống khỏi các rủi ro nghẽn cổ chai trong những khung giờ giao dịch mang tính chất quyết định.

- **Các bảng áp dụng:** CoSo (Cơ sở), Khoa (Khoa), HocPhan (Học phần),

ChuongTrinhDaoTao (Chương trình đào tạo), DotDangKy (Đợt đăng ký).

- **Lý do và lợi ích của việc áp dụng nhân bản toàn phần:**

- Tối ưu tốc độ truy xuất cục bộ: Việc đặt bản sao đầy đủ của năm bảng này ở tất cả 5 Server/Node giúp hệ thống tải danh sách môn học ngay lập tức, đáp ứng trải nghiệm người dùng theo thời gian thực mà không phụ thuộc vào độ trễ mạng.
- Giảm thiểu chi phí truyền tải mạng: Kỹ thuật này giúp loại bỏ hoàn toàn các phép kết mạng tốn kém (Network Joins) khi cần kết hợp dữ liệu giao dịch cục bộ (như LopHocPhan) với dữ liệu danh mục (như HocPhan) để lấy tên môn học.
- Tính sẵn sàng cao: Nhờ có nhiều bản sao độc lập, nếu mạng liên cơ sở gặp sự cố hoặc một Node bất kỳ bị sập, dữ liệu danh mục tại các Node còn lại vẫn hoàn toàn nguyên vẹn, đảm bảo hệ thống không bị tê liệt cục bộ.

3.4 Mô hình cấp phát dữ liệu

Giai đoạn cấp phát dữ liệu (Data Allocation) đóng vai trò ánh xạ các mảnh dữ liệu logic và các bản sao đã thiết kế ở những bước trước lên kiến trúc mạng vật lý của hệ thống. Dựa trên quy mô hoạt động của trường đại học, mô hình phân tán được triển khai thực tế trên năm node mạng độc lập, tương ứng với năm máy chủ đặt tại năm cơ sở đào tạo bao gồm Hòa Lạc, Ngọc Trục, Hà Đông, Cầu Giấy và Thành phố Hồ Chí Minh. Tại mỗi máy chủ vật lý này, hệ quản trị cơ sở dữ liệu sẽ đóng vai trò như một trạm tự trị, chịu trách nhiệm lưu trữ và xử lý phối hợp giữa luồng dữ liệu dùng chung và luồng dữ liệu tác nghiệp mang tính chất địa phương. Sự phân bổ này đảm bảo rằng nguồn dữ liệu luôn được đặt tại vị trí tối ưu nhất để phục vụ trực tiếp cho các giao dịch phát sinh từ chính cơ sở đó. Cấu trúc lưu trữ tại mỗi site được quy hoạch theo một mô hình hai tầng dữ liệu. Tầng thứ nhất là không gian lưu trữ dữ liệu nhân bản, nơi tất cả năm máy chủ đều được cấp phát một bản sao hoàn chỉnh và giống hệt nhau của ba bảng danh mục dùng chung bao gồm CoSo, Khoa, ChuongTrinhDaoTao, DotDangKy và HocPhan. Tầng thứ hai là không gian dữ liệu tác nghiệp cục bộ, nơi mỗi site chỉ tiếp nhận và lưu trữ các mảnh phân mảnh ngang của các bảng SinhVien, GiangVien, PhongHoc, LopHocPhan, LichHoc và DangKy thỏa mãn điều kiện trường dữ liệu khóa ngoại bám sát vào mã cơ sở quản lý của chính trạm đó. Mô hình cấp phát này mang lại những ưu thế về mặt hiệu năng xử lý và khả năng chịu lỗi cho toàn bộ kiến trúc phân tán.

Mỗi Site sẽ đóng vai trò là một Server Database hoàn toàn độc lập, lưu trữ trọn vẹn phần dữ liệu danh mục (được nhân bản) và phần dữ liệu nghiệp vụ sinh ra tại chính cơ sở đó. Cụ thể cấu trúc cấp phát như sau:

- **Site 1 (Hòa Lạc - Mã: HL):**

- Dữ liệu nhân bản: Chứa bản sao toàn phần của các bảng CoSo, Khoa,

ChuongTrinhDaoTao, DotDangKy, HocPhan.

- Dữ liệu cục bộ: Chứa mảnh phân mảnh ngang của các bảng SinhVien, GiangVien, PhongHoc, LopHocPhan thỏa mãn điều kiện `ID_headquarter = 'HL'`. Kéo theo đó là dữ liệu phân mảnh dẫn xuất của bảng LichHoc và DangKy thuộc các lớp học phần được mở tại cơ sở Hòa Lạc.

- **Site 2 (Ngọc Trục - Mã: NT):**

- Dữ liệu nhân bản: Chứa bản sao toàn phần của các bảng CoSo, Khoa, ChuongTrinhDaoTao, DotDangKy, HocPhan.
- Dữ liệu cục bộ: Chứa mảnh dữ liệu cục bộ và dẫn xuất tương ứng với các bản ghi có `ID_headquarter = 'NT'`.

- **Site 3 (Hà Đông - Mã: HD):**

- Dữ liệu nhân bản: Chứa bản sao toàn phần của các bảng CoSo, Khoa, ChuongTrinhDaoTao, DotDangKy, HocPhan.
- Dữ liệu cục bộ: Chứa mảnh dữ liệu cục bộ và dẫn xuất tương ứng với các bản ghi có `ID_headquarter = 'HD'`.

- **Site 4 (Cầu Giấy - Mã: CG):**

- Dữ liệu nhân bản: Chứa bản sao toàn phần của các bảng CoSo, Khoa, ChuongTrinhDaoTao, DotDangKy, HocPhan.
- Dữ liệu cục bộ: Chứa mảnh dữ liệu cục bộ và dẫn xuất tương ứng với các bản ghi có `ID_headquarter = 'CG'`.

- **Site 5 (TP. Hồ Chí Minh - Mã: HCM):**

- Dữ liệu nhân bản: Chứa bản sao toàn phần của các bảng CoSo, Khoa, ChuongTrinhDaoTao, DotDangKy, HocPhan.
- Dữ liệu cục bộ: Chứa mảnh dữ liệu cục bộ và dẫn xuất tương ứng với các bản ghi có `ID_headquarter = 'HCM'`.

4 XỬ LÝ GIAO DỊCH VÀ KIỂM SOÁT ĐỒNG THỜI

4.1 Bài toán tranh chấp dữ liệu (Lỗi Overbooking)

Trong thời gian đăng ký tín chỉ, một LopHocPhan môn CSDLPT chỉ còn đúng 1 chỗ trống (`number_of_student = 49`, `max_student = 50`). Cùng lúc đó (trong cùng 1 thời gian, cùng 1 mili giây), có 3 sinh viên nhấn nút đăng ký. Nguy cơ phát sinh lỗi: nếu hệ thống chỉ dùng câu lệnh `INSERT` thông thường, cả 3 luồng cùng đọc thấy số 49 (đáp ứng điều kiện < 50), và cùng thực hiện đăng ký thành công. Kết quả là lớp học có 52/50 sinh viên (Lỗi Overbooking / Lost Update).

4.2 Cơ chế xử lý đăng ký đồng thời

Để giải quyết triệt để bài toán trên và ngăn ngừa hiện tượng *Deadlock*, hệ thống áp dụng cơ chế quản trị giao dịch (Transaction) bằng kỹ thuật “Khóa mức dòng” và sắp xếp tài nguyên theo thứ tự cố định.

4.2.1 Sử dụng Transaction và khóa dòng (FOR UPDATE)

Mọi thao tác thay đổi dữ liệu đăng ký đều được đặt trong một khối Transaction liên mạch. Khi một giao dịch xử lý yêu cầu đăng ký cho sinh viên A, nó sẽ thực hiện truy vấn và chủ động khóa (Lock) bản ghi lớp học phần tương ứng bằng mệnh đề “FOR UPDATE”.

```
def _lock_classes_in_fixed_order(cursor, class_ids):
    ordered_ids = sorted(set(class_ids))
    cursor.execute(
        """
        SELECT id, semester, school_year, id_subject, number_of_student, max_student
        FROM lophocphan
        WHERE id = ANY(%s)
        ORDER BY id
        FOR UPDATE;
        """
    ,
    (ordered_ids,),
    )
    rows = cursor.fetchall()
    if len(rows) != len(ordered_ids):
        return None
    return [row[0]: row for row in rows]
```

Hệ thống sử dụng cơ chế khóa bản ghi (FOR UPDATE) để đảm bảo tính nhất quán dữ liệu khi nhiều sinh viên đăng ký học phần cùng lúc. Trước khi thực hiện đăng ký, các lớp học phần liên quan được khóa theo thứ tự ID cố định nhằm tránh hiện tượng deadlock giữa các transaction.


```

locked_classes = _lock_classes_in_fixed_order(cursor, class_ids_to_lock)
if locked_classes is None:
    conn.rollback()
    return False, "Không tìm thấy du lịch học phân liên quan"

target_class = locked_classes[class_id]
number_of_student, max_student = target_class[4], target_class[5]
if number_of_student >= max_student:
    conn.rollback()
    return False, "Lớp học phân đã đầy"

```

Sau khi khóa thành công, hệ thống kiểm tra số lượng sinh viên hiện tại (`number_of_student`) và số lượng tối đa cho phép (`max_student`) của lớp học phần. Nếu lớp đã đầy, transaction sẽ được hủy (ROLLBACK) và trả về thông báo lỗi. Nhờ đó, hệ thống ngăn chặn tình trạng đăng ký vượt quá sức chứa của lớp và đảm bảo dữ liệu luôn chính xác trong môi trường nhiều người dùng đồng thời.

4.2.2 Cơ chế phòng chống Deadlock

Deadlock xảy ra khi 2 giao dịch chờ đợi nhau giữ khóa tài nguyên (Ví dụ: Chức năng đổi lớp học phần yêu cầu hệ thống phải cập nhật số của 2 lớp cùng lúc: giảm lớp cũ, tăng lớp mới). Hệ thống loại bỏ hoàn toàn khả năng xảy ra Deadlock bằng chiến thuật: Sắp xếp thứ tự lấy khóa cố định.

```

def _lock_classes_in_fixed_order(cursor, class_ids):
    ordered_ids = sorted(set(class_ids))
    cursor.execute(
        """
        SELECT id, semester, school_year, id_subject, number_of_student, max_student
        FROM lophocphan
        WHERE id = ANY(%s)
        ORDER BY id
        FOR UPDATE;
        """
        ,
        (ordered_ids,)
    )
    rows = cursor.fetchall()
    if len(rows) != len(ordered_ids):
        return None
    return [(row[0]: row for row in rows)]

```

Bằng hàm `sorted()`, hệ thống luôn khóa lớp học phần có ID nhỏ trước, ID lớn sau. Việc các luồng luôn tuân thủ một thứ tự lấy tài nguyên giống nhau giúp loại trừ hoàn toàn việc tạo thành các chu trình khóa, từ đó đảm bảo hiệu năng và không bao giờ xảy ra tình trạng hệ thống bị treo cứng.

4.3 Minh họa kịch bản và kết quả mô phỏng

Để kiểm chứng tính đúng đắn của giải pháp, hệ thống đã cung cấp một công cụ mô phỏng trực quan trên giao diện Admin. Công cụ này tạo ra nhiều luồng (Threads) độc lập bắn yêu cầu HTTP song song vào hệ thống (mô phỏng nhiều sinh viên nhấn nút Đăng

ký cùng một mili-giây). Logic chạy Threading: Nhóm sử dụng thư viện `threading` của Python để giả lập gửi nhiều luồng request đăng ký vào hệ thống tại cùng 1 thời điểm.

Kết quả: Trong log hệ thống, chỉ có 1 thread thực hiện “SUCCESS”. Các threads còn lại đều nhận kết quả “FAILED: Lớp đã đầy”. Hệ thống giữ nguyên tính nhất quán của cột `number_of_student`.

Mô phỏng đăng ký đồng thời

Nhiều sinh viên cùng đăng ký một lớp có số chỗ giới hạn.

Site mở lớp

Cơ sở Hòa Lạc

Mã lớp học phần

LHP-HL-TEST

Số chỗ demo

1

Reset lớp test

Chạy mô phỏng

Danh sách sinh viên tham gia mô phỏng, mỗi dòng theo mẫu MA_SV,SITE

SV-HL-0002,HL
SV-HL-0003,HL
SV-HL-0004,HL
SV-HL-0005,HL

Tổng request

5

Thành công

1

Thất bại

4

Số chỗ tối đa

1

Kết quả từng transaction

id_student	id_student_headquarter	class_id	success	message	time
SV-HL-0001	HL	LHP-HL-TEST	☐	Lop hoc phan da day	2026-06-01 21:56:27.587627
SV-HL-0002	HL	LHP-HL-TEST	☒	Dang ky hoc phan thanh cong	2026-06-01 21:56:27.587993
SV-HL-0003	HL	LHP-HL-TEST	☐	Lop hoc phan da day	2026-06-01 21:56:27.588256
SV-HL-0004	HL	LHP-HL-TEST	☐	Lop hoc phan da day	2026-06-01 21:56:27.588515
SV-HL-0005	HL	LHP-HL-TEST	☐	Lop hoc phan da day	2026-06-01 21:56:27.588773

File ghi log đăng ký:

```
[2026-06-01 21:56:27] DANG_KY site=HL class=LHP-HL-TEST student=SV-HL-0002 success=True  
[2026-06-01 21:56:27] MO_PHONG_DONG_THOI class=LHP-HL-TEST success=1/5
```

5 TRUY VẤN PHÂN TÁN

5.1 Cơ chế thực thi truy vấn phân tán của hệ thống

Hệ thống gồm 5 cơ sở dữ liệu vật lý chạy độc lập. Để thực hiện được một “Truy vấn phân tán” – tức là lấy dữ liệu toàn trường, hệ thống sử dụng kiến trúc gộp dữ liệu tại tầng ứng dụng bằng công cụ Pandas DataFrame.

Quy trình thực thi chung:

- Backend (FastAPI) mở kết nối đồng thời tới cả 5 site.
- Gửi lệnh **SELECT** cục bộ xuống từng database để lấy dữ liệu thô.
- Chuyển đổi kết quả trả về từ mỗi site thành một DataFrame.
- lần lượt truy vấn từng site rồi tổng hợp bằng Pandas thông qua hàm `_all_sites`.

```
def _read_all_sites_query(query, params=None):  
    """Chạy cùng một truy vấn trên toàn bộ site rồi gom kết quả."""  
    frames = []  
    for site_code in SITE_CODES:  
        df = read_query(site_code, query, params)  
        if not df.empty:  
            frames.append(df)  
    return _concat_frames(frames)
```

- Áp dụng các thuật toán gom nhóm (GroupBy), lọc (Filter), sắp xếp (Sort) trên bộ nhớ (Memory) để ra được kết quả toàn cục.

5.2 Các truy vấn phân tán

5.2.1 Thống kê số lượng sinh viên đăng ký học phần theo từng cơ sở

Mục tiêu: đếm tổng số lượng sinh viên đã tham gia đăng ký học phần thành công, phân loại theo cơ sở mà sinh viên đó đang theo học.

Nguồn dữ liệu: truy vấn này lấy thông tin từ bảng `lophocphan` (đóng vai trò bảng chính để xác định cơ sở đang mở lớp) và kết hợp (**LEFT JOIN**) với bảng `dangky` (nhằm đếm số lượng sinh viên đã đăng ký hợp lệ).

Cách tổng hợp: Hệ thống thực hiện phép nối (**LEFT JOIN**) giữa bảng lớp học phần (`lophocphan`) và bảng đăng ký (`dangky`), kèm theo điều kiện lọc sinh viên có trạng thái `"DA_DANG_KY"`. Câu lệnh SQL tiếp tục đếm tổng số sinh viên (**COUNT**) và gom nhóm (**GROUP BY**) theo mã cơ sở quản lý (`id_headquarter`). Sau khi thu thập toàn bộ dữ liệu thô từ 5 site, tầng ứng dụng sử dụng phương thức `groupby` của Pandas dựa trên

trường mã và tên cơ sở (["id_headquarter", "site_name"]). Bằng hàm sum(), hệ thống cộng gộp số lượt đăng ký cục bộ thành một con số tổng duy nhất cho từng cơ sở. Cuối cùng, tập dữ liệu được sắp xếp theo thứ tự mã cơ sở trước khi trả về, giúp người quản trị dễ dàng đánh giá quy mô đào tạo của từng đơn vị trực thuộc.

5.2.2 Tìm kiếm học phần có nhiều sinh viên đăng ký nhất toàn trường

Mục tiêu: Xác định các môn học có nhiều sinh viên đăng ký nhiều nhất trên phạm vi cả 5 cơ sở cộng lại.

Nguồn dữ liệu: truy vấn này trích xuất dữ liệu từ ba bảng: bảng **hocphan** (để lấy định danh và tên môn học), bảng **lophocphan** (để xác định các lớp tương ứng đang được mở), và bảng **dangky** (để thống kê lượng sinh viên thực tế đã tham gia).

Cách tổng hợp: Truy vấn này thực hiện phép nối dữ liệu giữa ba bảng: học phần, lớp học phần và đăng ký. Ở mỗi site, hệ thống đếm số lượng bản ghi đăng ký hợp lệ tương ứng với từng mã học phần và trả về danh sách các học phần kèm theo tổng lượt đăng ký cục bộ. Khi dữ liệu được đưa lên tầng Pandas, hệ thống tiến hành thao tác gom nhóm lần hai (Secondary GroupBy) dựa trên mã và tên học phần. Lệnh sum() sẽ tính tổng số lượt đăng ký của một môn học trên cả 5 site cộng lại. Điểm mấu chốt của xử lý này là phương thức.sort_values("so_luot", ascending=False) nhằm sắp xếp toàn bộ danh sách theo chiều giảm dần của tổng số đăng ký. Qua đó, các học phần được đăng ký nhiều nhất sẽ lập tức hiển thị ở đầu bảng báo cáo.

5.2.3 Trích xuất danh sách sinh viên đăng ký chéo cơ sở

Mục tiêu: Trích xuất danh sách các sinh viên có cơ sở gốc quản lý khác với cơ sở nơi mở lớp học phần.

Nguồn dữ liệu: truy vấn này được lấy chủ yếu từ bảng **dangky** (nơi lưu trữ thông tin sinh viên cùng mã cơ sở gốc của họ) kết nối trực tiếp với bảng **lophocphan** (nơi chứa thông tin cơ sở đang tổ chức lớp học đó).

Cách tổng hợp: Để trích xuất danh sách này, hệ thống thực thi một câu lệnh truy vấn quét qua bảng **dangky** và **lophocphan**. Tại điều kiện WHERE, hệ thống đối chiếu trực tiếp mã cơ sở gốc của sinh viên (id_student_headquarter) với mã cơ sở tổ chức lớp (id_headquarter). Bất kỳ bản ghi nào có hai giá trị này khác nhau đều được xem là một trường hợp đăng ký chéo và được chọn ra kèm theo các thông tin thời gian đăng ký và trạng thái hiện tại. Vì thao tác lọc này đã được hoàn tất một cách tối ưu ngay tại Database Engine của từng site, tầng ứng dụng chỉ việc nối tất cả các bản ghi trả về từ 5 phân mảnh lại với nhau thành một bảng tổng hợp duy nhất mà không cần xử lý thêm các thuật toán phức tạp trên bộ nhớ.

5.2.4 Tính toán tỷ lệ lấp đầy của các lớp học phần toàn hệ thống

Mục tiêu: Đánh giá hiệu suất tối ưu hóa phòng học và nguồn lực giảng dạy thông qua tỷ lệ phần trăm giữa tổng số sinh viên thực tế trên tổng sức chứa tối đa.

Nguồn dữ liệu lấy từ bảng: được khai thác từ bảng **lophocphan** (để lấy số lượng sinh viên hiện tại và sức chứa tối đa của lớp) kết nối với bảng **hocphan** (để lấy và hiển thị tên môn học một cách trực quan).

Trong truy vấn này, hệ thống khai thác sức mạnh tính toán của SQL bằng cách thực hiện phép chia trực tiếp giữa số sinh viên hiện tại (**number_of_student**) và sức chứa tối đa (**max_student**). Để tránh lỗi chia cho không (Divide by zero) nếu bảng ghi cấu hình lỗi, hệ thống áp dụng hàm **NULLIF(max_student, 0)**. Số liệu phần trăm được nhân cho 100 và làm tròn đến hai chữ số thập phân thông qua hàm **ROUND**. Đồng thời, hệ thống cũng trích xuất trực tiếp số chỗ còn trống (**max_student - number_of_student**). Việc đưa công thức toán học vào thẳng câu truy vấn SQL giúp tiết kiệm đáng kể tài nguyên xử lý ở tầng backend. DataFrame cuối cùng thu được sẽ cung cấp bức tranh toàn cảnh về độ quá tải hay trống trải của từng lớp học trên khắp các cơ sở.

5.2.5 Thống kê quy mô lớp học và số lượng sinh viên theo cơ sở

Mục tiêu: nắm bắt được quy mô đào tạo (số lớp học phần được mở) của mỗi cơ sở.

Nguồn dữ liệu: hệ thống truy xuất dữ liệu trực tiếp từ một bảng duy nhất là **lophocphan** để đếm tổng số bản ghi (**COUNT(id)**) được gom theo mã cơ sở. Tương tự, đối với thống kê quy mô sinh viên, câu lệnh chỉ quét qua bảng **sinhvien** và nhóm lại để đếm tổng số.

Để đánh giá tải trọng hành chính và quản lý thực tế, hệ thống cung cấp thêm hai báo cáo phân tán song song là thống kê số lớp học phần đang mở và thống kê tổng lượng sinh viên thuộc quyền quản lý của mỗi đơn vị.

Đối với số lớp học phần, truy vấn SQL đơn giản là đếm tổng số bản ghi (**COUNT(id)**) trong bảng **lophocphan** được gom theo mã cơ sở. Tương tự, để thống kê sinh viên, câu lệnh quét qua bảng **sinhvien** và nhóm lại để đếm số. Mặc dù các phép đếm cục bộ được thực hiện độc lập tại các site phân mảnh, bước gom nhóm cuối cùng trên Pandas DataFrame thông qua hàm **sum()** sẽ ráp nối những mảnh ghép này lại. Qua đó, nhà trường có thể xuất ra những báo cáo ma trận chính xác về sự tương quan giữa số lượng sinh viên đang theo học và số lượng lớp học phần phải mở tại một thời điểm, làm cơ sở cho việc điều phối giảng viên toàn trường.

6 YÊU CẦU NÂNG CAO

Trong quá trình xây dựng hệ thống đăng ký học phần phân tán, nhóm triển khai một số nội dung nâng cao nhằm mô phỏng các vấn đề thường gặp trong hệ cơ sở dữ liệu phân tán. Các nội dung bao gồm kiểm soát đăng ký đồng thời, phòng tránh deadlock, nhân bản dữ liệu dùng chung và xử lý khi một site tạm thời mất kết nối. Các nội dung này giúp hệ thống thể hiện rõ đặc trưng của cơ sở dữ liệu phân tán: dữ liệu được phân mảnh theo cơ sở, dữ liệu dùng chung được nhân bản, các thao tác ghi cần đảm bảo tính nhất quán, còn các thao tác đọc ưu tiên tính sẵn sàng.

Nội dung nâng cao	Mục tiêu áp dụng trong hệ thống
Xung đột cập nhật và deadlock	Đảm bảo nhiều sinh viên đăng ký đồng thời không làm vượt sĩ số, đồng thời giảm nguy cơ deadlock khi đổi lớp chéo.
Nhân bản dữ liệu dùng chung	Giúp các site đọc nhanh dữ liệu như cơ sở, khoa, học phần, chương trình đào tạo và đợt đăng ký.
Site tạm thời mất kết nối	Không ghi tạm giao dịch đăng ký khi thiếu dữ liệu kiểm tra, nhưng vẫn lưu yêu cầu để xử lý lại sau.

Bảng 1: Các nội dung nâng cao triển khai trong hệ thống

6.1 Mô phỏng xung đột cập nhật và phòng tránh deadlock

6.1.1 Bối cảnh mô phỏng

Trong hệ thống đăng ký học phần, nhiều sinh viên có thể đăng ký cùng một lớp học phần tại cùng một thời điểm. Nếu không có cơ chế kiểm soát đồng thời, các transaction có thể cùng đọc thấy lớp còn chỗ và cùng ghi dữ liệu đăng ký, dẫn đến sĩ số vượt quá giới hạn cho phép. Ví dụ, lớp học phần còn 1 chỗ nhưng có 2 sinh viên gửi yêu cầu đăng ký đồng thời. Nếu cả hai giao dịch đều đọc được số lượng còn trống trước khi cập nhật, hệ thống có thể ghi nhận cả hai đăng ký và làm `number_of_student` vượt quá `max_student`.

6.1.2 Nguy cơ phát sinh lỗi dữ liệu

- Sĩ số lớp học phần vượt quá `max_student`.
- Bảng `DangKy` ghi nhận đăng ký thành công nhưng bảng `LopHocPhan` không cập nhật đúng số lượng.
- Một sinh viên gửi nhiều request song song vào cùng một học phần gây trùng hoặc lệch trạng thái.
- Hai transaction khóa tài nguyên theo thứ tự khác nhau có thể tạo deadlock.

6.1.3 Cơ chế kiểm soát đồng thời

Để xử lý vấn đề trên, hệ thống sử dụng transaction kết hợp với khóa dòng trên bảng LopHocPhan. Khi sinh viên đăng ký, backend khóa lớp học phần bằng `SELECT ... FOR UPDATE`, kiểm tra sĩ số hiện tại, sau đó mới ghi dữ liệu vào bảng DangKy. Câu lệnh `SELECT ... FOR UPDATE` giúp khóa dòng lớp học phần đang được xử lý. Trong thời gian transaction thứ nhất chưa commit hoặc rollback, transaction khác muốn cập nhật cùng lớp học phần phải chờ. Cách này giúp tránh tình huống nhiều request cùng tăng sĩ số tại một thời điểm.

Ngoài ra, hệ thống sử dụng `pg_advisory_xact_lock()` để khóa logic theo cặp sinh viên và học phần. Cơ chế này giúp tránh việc nhiều request của cùng một sinh viên đối với cùng một học phần chạy song song và gây xung đột.

```
# dùng để chống việc 2 request đăng ký cùng một sinh viên + cùng một môn/lớp liên quan chạy song song gây xung đột.
cursor.execute(
    "SELECT pg_advisory_xact_lock(hashtext(%s));",
    (f"{student_id}:{_subject_key(target_class)}",),
)
```

Hình 1: Sử dụng `pg_advisory_xact_lock` để khóa logic request trùng

Đối với tình huống đổi lớp chéo, ví dụ sinh viên A đổi từ lớp 1 sang lớp 2, đồng thời sinh viên B đổi từ lớp 2 sang lớp 1, hệ thống có nguy cơ phát sinh deadlock nếu mỗi transaction khóa lớp theo thứ tự khác nhau. Để phòng tránh, hệ thống gom các lớp liên quan, sắp xếp mã lớp theo thứ tự cố định, sau đó khóa các lớp theo cùng một thứ tự.

Cơ chế	Vai trò
Transaction	Gom toàn bộ thao tác đăng ký thành một đơn vị xử lý nguyên tử.
<code>SELECT ... FOR UPDATE</code>	Khóa dòng lớp học phần để kiểm soát cập nhật đồng thời.
<code>pg_advisory_xact_lock</code>	Khóa logic theo sinh viên và học phần, tránh request trùng chạy song song.
Khóa lớp theo thứ tự cố định	Giảm nguy cơ deadlock trong tình huống đổi lớp chéo.
Trigger atomic update	Tăng sĩ số chỉ khi <code>number_of_student < max_student</code> .
CHECK constraint	Ngăn dữ liệu sai ở tầng cơ sở dữ liệu.

Bảng 2: Các cơ chế kiểm soát đồng thời được áp dụng

6.1.4 Vai trò của trigger trong kiểm soát sĩ số

Bên cạnh xử lý ở backend, trigger trên bảng DangKy tự động cập nhật sĩ số lớp học phần. Việc tăng sĩ số được thực hiện bằng atomic update, tức là điều kiện còn chỗ được

đặt trực tiếp trong câu UPDATE. Nếu lớp đã đầy, trigger phát sinh lỗi và giao dịch bị rollback.

```
UPDATE LopHocPhan
SET number_of_student = number_of_student + 1
WHERE ID = NEW.ID_class
AND number_of_student < max_student;
```

Hình 2: Atomic update số chỗ bên trong Trigger

Nhờ cách này, ngay cả khi nhiều transaction chạy đồng thời, tăng cơ sở dữ liệu vẫn có kiểm tra cuối cùng để đảm bảo `number_of_student` không vượt quá `max_student`.

6.1.5 Kết quả và nhận xét

Khi mô phỏng nhiều sinh viên đăng ký đồng thời vào một lớp học phần có số chỗ giới hạn, hệ thống chỉ cho phép số lượng đăng ký thành công tương ứng với số chỗ còn lại. Các request còn lại bị từ chối với thông báo lớp học phần đã đầy. Điều này cho thấy hệ thống ưu tiên consistency (tính nhất quán) đối với nghiệp vụ đăng ký học phần.

6.2 Nhân bản dữ liệu dùng chung

6.2.1 Lý do cần nhân bản

Trong hệ thống đăng ký học phần phân tán, một số dữ liệu được sử dụng chung cho toàn trường như `CoSo`, `Khoa`, `HocPhan`, `ChuongTrinhDaoTao` và `DotDangKy`. Đây là các dữ liệu có tần suất cập nhật không cao nhưng được nhiều site sử dụng thường xuyên để tra cứu, mở lớp học phần và kiểm tra điều kiện đăng ký. Nếu mỗi lần cần sử dụng các dữ liệu này đều phải truy vấn sang site khác, hệ thống sẽ phụ thuộc nhiều vào kết nối liên site, làm tăng độ trễ và giảm tính sẵn sàng.

Vì vậy, hệ thống sử dụng `logical replication` của PostgreSQL để nhân bản dữ liệu dùng chung. Trong mô hình này, site `site_hadong` đóng vai trò `publisher`, các site còn lại đóng vai trò `subscriber`. Khi dữ liệu dùng chung được cập nhật tại site nguồn, PostgreSQL sẽ tự động đồng bộ thay đổi sang các site còn lại.

6.2.2 Các bảng được nhân bản

Bảng dữ liệu	Ý nghĩa	Lý do nhân bản
CoSo	Thông tin các cơ sở đào tạo	Dùng chung để xác định site và cơ sở quản lý.
Khoa	Thông tin khoa/ngành	Cần cho sinh viên, giảng viên, học phần và đợt đăng ký.
HocPhan	Danh mục học phần	Sinh viên ở mọi site đều cần tra cứu.
ChuongTrinhDaoTao	Chương trình đào tạo theo khoa/ngành	Dùng để kiểm tra học phần thuộc chương trình đào tạo.
DotDangKy	Đợt đăng ký học phần	Dùng để xác định thời gian và phạm vi được phép đăng ký.

Bảng 3: Danh sách các bảng dữ liệu được nhân bản

6.2.3 Lợi ích của replication

Việc nhân bản dữ liệu dùng chung mang lại một số lợi ích chính:

- Các site có thể đọc dữ liệu ngay tại local database.
- Giảm số lượng truy vấn liên site.
- Tăng tốc độ tra cứu học phần, khoa, chương trình đào tạo và đợt đăng ký.
- Tăng availability (tính sẵn sàng) cho các thao tác đọc dữ liệu.
- Giảm phụ thuộc vào một site trung tâm.
- Phù hợp với các bảng dữ liệu danh mục có tần suất cập nhật thấp.

Nhờ đó, các thao tác đọc như tra cứu học phần, xem chương trình đào tạo hoặc kiểm tra đợt đăng ký có thể thực hiện nhanh hơn và ổn định hơn tại từng site.

6.2.4 Hạn chế và cách lựa chọn dữ liệu nhân bản

Replication giúp tăng hiệu năng và tính sẵn sàng, nhưng cũng có hạn chế là dữ liệu giữa các site có thể bị lệch tạm thời trong một khoảng thời gian ngắn. Vì vậy, đối với dữ liệu dùng chung, hệ thống chấp nhận mô hình *eventual consistency* (nhất quán cuối cùng). Điều này có nghĩa là khi dữ liệu được cập nhật tại site nguồn, các site khác có thể chưa nhận được thay đổi ngay lập tức. Tuy nhiên, sau quá trình đồng bộ, các bản sao sẽ đạt trạng thái nhất quán.

Hệ thống không nhân bản toàn bộ dữ liệu, mà lựa chọn theo đặc điểm từng loại dữ liệu:

Loại dữ liệu	Chiến lược xử lý
Dữ liệu danh mục dùng chung	Nhân bản bằng logical replication, ưu tiên availability khi đọc.
Dữ liệu đăng ký học phần	Không nhân bản ghi chính, ưu tiên consistency khi ghi.
Dữ liệu sĩ số lớp học phần	Cập nhật tại site mở lớp, kiểm soát bằng transaction, lock và trigger.

Bảng 4: Chiến lược xử lý dữ liệu nhân bản

Việc không nhân bản bảng **DangKy** theo cơ chế nhiều site cùng ghi là cần thiết, vì đây là dữ liệu giao dịch có tần suất thay đổi cao và ảnh hưởng trực tiếp đến sĩ số lớp học phần. Nếu cho phép nhiều site cùng ghi rồi đồng bộ sau, hệ thống dễ phát sinh xung đột, sai lệch sĩ số hoặc khó kiểm soát thứ tự transaction.

6.3 Cơ chế xử lý khi một site tạm thời mất kết nối

6.3.1 Bối cảnh lỗi site

Trong hệ thống phân tán, việc một site tạm thời mất kết nối là tình huống có thể xảy ra. Đối với nghiệp vụ đăng ký học phần, hệ thống không nên ghi dữ liệu khi chưa kiểm tra đầy đủ các điều kiện nghiệp vụ như sĩ số lớp, trùng lịch, trạng thái đợt đăng ký và chương trình đào tạo. Nếu hệ thống vẫn ghi tạm vào bảng **DangKy** trong khi chưa kiểm tra được dữ liệu ở site liên quan, dữ liệu có thể bị sai lệch. Vì vậy, hệ thống chọn hướng ưu tiên consistency: không ghi tạm giao dịch đăng ký khi thiếu dữ liệu kiểm tra, nhưng lưu yêu cầu để xử lý lại sau.

6.3.2 Bảng **OfflineOperationLog**

Hệ thống bổ sung cơ chế ghi nhận yêu cầu bị gián đoạn bằng bảng **OfflineOperationLog**. Bảng này được lưu tại site điều phối HD. Khi một thao tác đăng ký hoặc hủy đăng ký gặp lỗi do site mất kết nối, hệ thống không ghi tạm vào bảng **DangKy** mà lưu yêu cầu vào **OfflineOperationLog** với trạng thái PENDING.

Trạng thái	Ý nghĩa
PENDING	Yêu cầu đang chờ xử lý lại.
RETRYING	Hệ thống hoặc quản trị viên đang thử xử lý lại yêu cầu.
DONE	Retry thành công, yêu cầu đã được xử lý.
FAILED	Retry thất bại do điều kiện nghiệp vụ không còn thỏa mãn hoặc vẫn lỗi.
CANCELLED	Quản trị viên hủy yêu cầu chờ xử lý.

Bảng 5: Các trạng thái của yêu cầu chờ xử lý offline

6.3.3 Luồng xử lý khi site mất kết nối

Quy trình xử lý được diễn ra theo các bước sau:

1. Người dùng gửi yêu cầu đăng ký hoặc hủy đăng ký học phần.
2. Hệ thống kiểm tra các điều kiện nghiệp vụ cần thiết.
3. Nếu các site cần thiết đều online, hệ thống xử lý và commit như bình thường.
4. Nếu một site cần thiết mất kết nối, hệ thống rollback transaction nếu có.
5. Yêu cầu được lưu vào `OfflineOperationLog` với trạng thái PENDING.
6. Khi site hoạt động trở lại, quản trị viên vào giao diện Yêu cầu chờ xử lý và bấm Thử lại (Retry).
7. Khi retry, hệ thống gọi lại logic đăng ký hoặc hủy đăng ký hiện có và kiểm tra lại toàn bộ điều kiện nghiệp vụ.
8. Nếu retry thành công, trạng thái chuyển thành DONE; nếu thất bại, trạng thái chuyển thành FAILED và lưu lý do lỗi.

6.3.4 Đánh giá cơ chế xử lý offline

Cơ chế này thể hiện định hướng ưu tiên tính nhất quán dữ liệu. Hệ thống không cố ghi tạm giao dịch khi thiếu dữ liệu kiểm tra từ site khác, nhưng vẫn đảm bảo yêu cầu của người dùng không bị mất và có thể xử lý lại sau. Với các thao tác đọc dữ liệu, hệ thống có thể ưu tiên availability và sử dụng dữ liệu nhân bản local nếu có. Tuy nhiên, với thao tác ghi như đăng ký học phần, hệ thống ưu tiên consistency để tránh vượt số, trùng lịch hoặc đăng ký sai điều kiện.

6.4 Kết luận

Thông qua các nội dung nâng cao trên, hệ thống đã mô phỏng và xử lý được nhiều vấn đề quan trọng trong hệ cơ sở dữ liệu phân tán. Hệ thống có cơ chế kiểm soát đồng thời để tránh vượt sĩ số, phòng tránh deadlock khi đổi lớp chéo, nhân bản dữ liệu dùng chung để giảm phụ thuộc truy vấn liên site, đồng thời bổ sung cơ chế ghi nhận và xử lý lại yêu cầu khi một site tạm thời mất kết nối. Các cơ chế này giúp hệ thống không chỉ đáp ứng nghiệp vụ đăng ký học phần nhiều cơ sở, mà còn thể hiện rõ các đặc trưng của hệ cơ sở dữ liệu phân tán như phân mảnh, nhân bản, kiểm soát đồng thời và xử lý lỗi site.

7 TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Chương này trình bày kiến trúc triển khai, công nghệ sử dụng và đánh giá hệ thống đăng ký học phần nhiều cơ sở. Nội dung tập trung vào ba khía cạnh chính: kiến trúc hệ thống, tính đúng đắn và hiệu năng, đồng thời đánh giá cơ chế xử lý lỗi khi một site tạm thời mất kết nối.

7.1 Kiến trúc hệ thống và công nghệ sử dụng

Hệ thống đăng ký học phần nhiều cơ sở được triển khai theo mô hình ứng dụng web kết hợp cơ sở dữ liệu phân tán. Mỗi cơ sở đào tạo tương ứng với một site PostgreSQL riêng, chịu trách nhiệm lưu trữ và xử lý dữ liệu cục bộ của cơ sở đó. Các site cùng tham gia vào một hệ thống chung thông qua backend API và giao diện người dùng.

Kiến trúc tổng quát của hệ thống được mô tả như sau:

Streamlit Frontend → FastAPI Backend → PostgreSQL Sites

Thành phần/công nghệ	Vai trò trong hệ thống
Streamlit Frontend	Cung cấp giao diện cho quản trị viên, sinh viên và giảng viên.
FastAPI Backend	Xử lý logic nghiệp vụ, xác thực, phân quyền, đăng ký/hủy đăng ký học phần, truy vấn phân tán và xử lý lỗi kết nối site.
PostgreSQL	Lưu trữ dữ liệu tại từng site. Mỗi site là một PostgreSQL database riêng.
Docker Compose	Khởi tạo và quản lý nhiều container PostgreSQL tương ứng với nhiều cơ sở đào tạo.
Pandas	Tổng hợp dữ liệu khi thực hiện truy vấn phân tán từ nhiều site.

Bảng 6: Các công nghệ sử dụng trong hệ thống

Hệ thống hiện có 5 site PostgreSQL:

Mã site	Cơ sở	Database	Port
HL	Hòa Lạc	site_hoalac	5440
NT	Ngọc Trục	site_ngoctruc	5441
HD	Hà Đông	site_hadong	5442
CG	Cầu Giấy	site_caugiay	5443
HCM	TP. Hồ Chí Minh	site_hcm	5444

Bảng 7: Danh sách các site phân tán

Dữ liệu trong hệ thống được tổ chức theo hướng phân mảnh ngang theo cơ sở. Các bảng nghiệp vụ như *SinhVien*, *GiangVien*, *PhongHoc*, *LopHocPhan*, *LichHoc* và *DangKy* được lưu tại site tương ứng với cơ sở quản lý hoặc cơ sở mở lớp. Cách tổ chức này giúp mỗi site xử lý dữ liệu cục bộ của mình độc lập hơn.

Bên cạnh dữ liệu cục bộ, một số bảng dùng chung toàn trường được nhân bản giữa các site, bao gồm:

- *CoSo*
- *Khoa*
- *HocPhan*
- *ChuongTrinhDaoTao*
- *DotDangKy*

Các bảng này phục vụ cho việc tra cứu thông tin chung như học phần, khoa/ngành, chương trình đào tạo và đợt đăng ký. Việc nhân bản giúp giảm nhu cầu truy vấn liên site đối với dữ liệu dùng chung.

Backend được chia thành nhiều lớp nhằm tăng khả năng bảo trì và mở rộng:

Lớp/thư mục	Chức năng
<i>api</i>	Định nghĩa các endpoint FastAPI.
<i>services</i>	Xử lý nghiệp vụ chính.
<i>db</i>	Quản lý kết nối và truy vấn database.
<i>models</i>	Định nghĩa schema dữ liệu request.
<i>frontend/streamlit_app</i>	Chứa giao diện Streamlit.

Bảng 8: Cấu trúc phân lớp của Backend

Nhờ cách chia lớp này, hệ thống dễ mở rộng khi bổ sung các chức năng như truy vấn phân tán, mô phỏng đăng ký đồng thời, replication hoặc xử lý site mất kết nối.

7.2 Đánh giá tính đúng đắn và hiệu năng của hệ thống

Về tính đúng đắn, hệ thống được thiết kế để đảm bảo các nghiệp vụ đăng ký học phần tuân thủ đầy đủ các ràng buộc cần thiết. Khi sinh viên đăng ký một lớp học phần, backend thực hiện các bước kiểm tra sau:

- Kiểm tra trạng thái đăng ký học phần đang mở hay đóng.
- Xác định sinh viên tồn tại tại site quản lý hồ sơ.
- Lấy khoa/ngành và năm nhập học của sinh viên.
- Kiểm tra lớp học phần tồn tại tại site mở lớp.
- Kiểm tra có đợt đăng ký hợp lệ theo học kỳ, năm học, khoa/ngành và khóa tuyển sinh.
- Kiểm tra học phần thuộc chương trình đào tạo của sinh viên.
- Kiểm tra lớp còn chỗ.
- Kiểm tra trùng lịch học.
- Nếu sinh viên đã đăng ký lớp khác cùng học phần, hệ thống xử lý như nghiệp vụ đổi lớp.
- Ghi dữ liệu vào bảng `DangKy` và cập nhật số.

Hệ thống sử dụng transaction để đảm bảo một thao tác đăng ký hoặc hủy đăng ký chỉ được commit khi toàn bộ điều kiện hợp lệ. Nếu có lỗi xảy ra, transaction sẽ rollback để tránh dữ liệu bị ghi dở.

Để kiểm soát đăng ký đồng thời, hệ thống sử dụng nhiều cơ chế:

- `SELECT ... FOR UPDATE` để khóa dòng lớp học phần.
- `pg_advisory_xact_lock()` để khóa logic theo sinh viên và học phần.
- Khóa các lớp liên quan theo thứ tự cố định khi đổi lớp.
- Trigger trên bảng `DangKy` để cập nhật số.
- Atomic update để tránh vượt quá số tối đa.

Ví dụ trigger tăng số chỉ thực hiện khi lớp còn chỗ:

```
UPDATE LopHocPhan
SET number_of_student = number_of_student + 1
WHERE ID = NEW.ID_class
AND number_of_student < max_student;
```

Hình 3: Đoạn lệnh cập nhật số an toàn

Cách xử lý này giúp tránh tình huống nhiều transaction cùng đọc thấy lớp còn chỗ rồi cùng tăng sĩ số, dẫn đến vượt quá `max_student`. Hệ thống cũng có mô phỏng đăng ký đồng thời. Trong kịch bản này, nhiều sinh viên cùng đăng ký một lớp có số chỗ giới hạn. Kết quả mong đợi là chỉ số lượng sinh viên hợp lệ được đăng ký, các request còn lại bị từ chối khi lớp đầy. Điều này cho thấy hệ thống xử lý đúng trong trường hợp có cạnh tranh tài nguyên.

Ngoài ra, hệ thống có mô phỏng đổi lớp chéo để kiểm tra khả năng phòng tránh deadlock. Khi hai sinh viên đổi lớp ngược chiều cùng lúc, hệ thống khóa các lớp liên quan theo thứ tự cố định. Nhờ đó, các transaction không tạo thành vòng chờ và giảm nguy cơ deadlock.

Về hiệu năng, hệ thống có một số điểm thuận lợi:

- Dữ liệu nghiệp vụ được phân mảnh theo cơ sở, giúp thao tác cục bộ không phải truy cập toàn bộ hệ thống.
- Các bảng dùng chung được replication, giúp mỗi site đọc dữ liệu chung tại local database.
- Truy vấn phân tán được thực hiện bằng cách đọc từng site rồi tổng hợp kết quả bằng Pandas.
- Các thao tác đăng ký sử dụng khóa ở phạm vi hẹp, chủ yếu trên dòng lớp học phân và bản ghi đăng ký liên quan.

Tuy nhiên, hệ thống vẫn có một số giới hạn:

- Một số kiểm tra nghiệp vụ, đặc biệt là kiểm tra trùng lịch toàn hệ thống, có thể cần truy cập nhiều site.
- Nếu một site mất kết nối, hệ thống ưu tiên tính nhất quán nên có thể chưa cho phép đăng ký ngay.
- Logical replication hiện được áp dụng cho dữ liệu dùng chung, chưa áp dụng cho dữ liệu giao dịch như `DangKy`.

Nhìn chung, hệ thống đảm bảo tốt tính đúng đắn nghiệp vụ và có mức hiệu năng phù hợp với phạm vi mô phỏng đồ án. Việc phân mảnh theo cơ sở và nhân bản dữ liệu dùng chung giúp giảm tải cho các truy vấn không cần thiết giữa các site.

7.3 Đánh giá xử lý lỗi và kịch bản kết nối

Trong hệ cơ sở dữ liệu phân tán, một site có thể tạm thời mất kết nối do lỗi mạng, container bị dừng hoặc database chưa sẵn sàng. Vì vậy, project có triển khai cơ chế kiểm tra trạng thái site và xử lý lỗi kết nối trong quá trình thực hiện nghiệp vụ.

Hệ thống có chức năng kiểm tra kết nối từng site. Backend thử mở kết nối đến từng PostgreSQL site và trả về trạng thái `OK` hoặc `ERROR`. Thông tin này được hiển thị trên

dashboard admin, giúp quản trị viên biết site nào đang hoạt động và site nào đang lỗi. Đối với truy vấn phân tán, hệ thống có cơ chế ghi nhận các site lỗi. Khi một truy vấn được thực hiện trên nhiều site, nếu một site không phản hồi, hệ thống ghi log lỗi và có thể trả cảnh báo rằng kết quả chưa bao gồm site đó. Cách này giúp người dùng hiểu rằng dữ liệu báo cáo có thể chưa đầy đủ trong thời điểm site bị lỗi.

Đối với nghiệp vụ đăng ký học phần, hệ thống chọn hướng ưu tiên tính nhất quán dữ liệu. Đăng ký học phần là nghiệp vụ có nhiều ràng buộc liên site như trùng lịch, sĩ số lớp, trạng thái đợt đăng ký và chương trình đào tạo. Nếu một site cần thiết bị mất kết nối, hệ thống không ghi tạm dữ liệu vào `DangKy`, vì điều đó có thể gây sai lệch dữ liệu. Thay vào đó, project bổ sung cơ chế `OfflineOperationLog`. Đây là bảng dùng để lưu lại các yêu cầu đăng ký hoặc hủy đăng ký bị gián đoạn do site mất kết nối.

Bảng `OfflineOperationLog` lưu các thông tin chính:

Thuộc tính	Ý nghĩa
<code>site_code</code>	Site bị lỗi hoặc site liên quan đến yêu cầu.
<code>action</code>	Thao tác bị lỗi, ví dụ <code>DANG_KY</code> hoặc <code>HUY_DANG_KY</code> .
<code>payload</code>	Dữ liệu cần thiết để xử lý lại yêu cầu.
<code>error_message</code>	Thông báo lỗi tại thời điểm xảy ra sự cố.
<code>status</code>	Trạng thái xử lý của yêu cầu.
<code>created_at</code>	Thời điểm yêu cầu được ghi nhận.
<code>retried_at</code>	Thời điểm thử xử lý lại gần nhất.
<code>retry_count</code>	Số lần đã thử xử lý lại.

Bảng 9: Cấu trúc bảng `OfflineOperationLog`

Các trạng thái của yêu cầu offline gồm:

Trạng thái	Ý nghĩa
<code>PENDING</code>	Yêu cầu đã được ghi nhận và chờ xử lý lại.
<code>RETRYING</code>	Hệ thống đang thử xử lý lại.
<code>DONE</code>	Retry thành công.
<code>FAILED</code>	Retry thất bại.
<code>CANCELLED</code>	Admin hủy yêu cầu.

Bảng 10: Các trạng thái lưu vết xử lý ngoại tuyến

Luồng xử lý khi site mất kết nối:

1. Người dùng gửi yêu cầu đăng ký/hủy đăng ký.

2. Hệ thống thực hiện kiểm tra nghiệp vụ.

3. Nếu site cần thiết mất kết nối:

- Rollback transaction nếu có.
- Không ghi dữ liệu vào bảng `DangKy`.
- Lưu yêu cầu vào `OfflineOperationLog`.
- Đặt trạng thái = `PENDING`.

Khi site hoạt động trở lại, admin vào giao diện “Yêu cầu chờ xử lý” và bấm “Thử lại”. Khi retry, hệ thống không ghi trực tiếp theo dữ liệu cũ, mà gọi lại logic đăng ký/hủy đăng ký hiện có. Nhờ đó, toàn bộ điều kiện nghiệp vụ được kiểm tra lại tại thời điểm retry. Nếu retry thành công, yêu cầu chuyển sang trạng thái `DONE`. Nếu lớp đã đầy, đợt đăng ký đã đóng, sinh viên bị trùng lịch hoặc site vẫn chưa kết nối được, yêu cầu chuyển sang trạng thái `FAILED` và lưu lại lý do lỗi.

Cơ chế này có ưu điểm là không làm mất yêu cầu của người dùng khi site gặp sự cố, đồng thời vẫn đảm bảo dữ liệu cuối cùng được ghi một cách hợp lệ. Đây là lựa chọn phù hợp với hệ thống đăng ký học phần, vì tính đúng đắn dữ liệu quan trọng hơn việc ghi tạm khi chưa kiểm tra đủ điều kiện. Tuy vậy, cơ chế này cũng có giới hạn. Khi một site mất kết nối, hệ thống không đảm bảo đăng ký thành công ngay lập tức nếu còn phụ thuộc vào dữ liệu từ site đó. Đây là đánh đổi giữa tính nhất quán và tính sẵn sàng. Trong phạm vi hệ thống, nhóm ưu tiên tính nhất quán dữ liệu để tránh các lỗi nghiêm trọng như trùng lịch hoặc vượt sĩ số.

Tổng kết lại, hệ thống đã có các cơ chế xử lý lỗi chính:

- Kiểm tra trạng thái kết nối từng site.
- Ghi log khi site lỗi.
- Rollback transaction khi thao tác ghi thất bại.
- Lưu yêu cầu lỗi vào `OfflineOperationLog`.
- Cho phép admin retry khi site hoạt động trở lại.
- Kiểm tra lại điều kiện nghiệp vụ trước khi ghi dữ liệu.

Nhờ vậy, hệ thống có khả năng xử lý các kịch bản lỗi kết nối ở mức an toàn và phù hợp với yêu cầu của một hệ cơ sở dữ liệu phân tán mô phỏng.